Student Management API

CPEN 208 - Project 2 Report

Computer Engineering Department Student Management System

This report is based on the implemented project source code, database scripts, and README. It describes only functionality and verification evidenced in the repository.

Implementation: Next.js 14 application with PostgreSQL
Deliverable: REST-style JSON web service and authenticated student/admin portal

1. Introduction and objective

The Student Management System supports student records, courses, teaching assignments, fees, and payments for the Computer Engineering Department. Project 1 established the PostgreSQL database functionality. Project 2 exposes that functionality through Next.js App Router API route handlers under src/app/api.
The objective is to provide JSON web-service endpoints used by the existing student portal and administrator interface while enforcing role-based access. Students can retrieve only their own personal, enrolment, fee, payment, and outstanding-balance data. Administrators can retrieve department-wide and individual student data.

2. Technologies used

- Next.js 14.2.35 App Router for pages, dashboards, and API route handlers.
- React 18 and JavaScript for the student and administrator interfaces.
- Node.js server runtime; pg PostgreSQL driver for the shared connection pool.
- PostgreSQL database named cpen208_project.
- bcrypt for password hashing and login verification.

3. Database design and Project 1 functions

The schema contains students, courses, enrollments, lecturers, teaching_assistants, course_lecturers, lecturer_tas, fees, payments, and users. The course_lecturers and lecturer_tas junction tables represent teaching assignments. The users table stores student/admin roles and password hashes.
- Student personal information: students table; exposed by /api/me and /api/students/{student_id}.
- Student fees/payments: fees and payments tables; exposed by my-* and student-specific endpoints.
- Course enrollment: enrollments joined to courses; exposed by /api/my-courses and /api/students/{student_id}/courses.
- Lecturer-to-course assignment: course_lecturers; exposed by /api/courses/{course_id}/lecturer.
- Lecturer-to-TA assignment: lecturer_tas; exposed by /api/lecturers/{lecturer_id}/teaching-assistants.

Two calculate_outstanding_fees database functions are included: a department-wide overload and a student-ID overload. The student endpoint uses the parameterized student-ID overload.

4. Project 2 API/web-service implementation

Each API endpoint is a Next.js route.js handler. Shared modules provide the PostgreSQL pool, JSON response helpers, ID validation, signed session handling, and authorization guards. Data routes return JSON success/data envelopes; authentication routes return JSON message or error values.
Status codes implemented are 200 for successful reads, 201 for successful registration, 400 for invalid input, 401 for missing authentication, 403 for role or ownership violations, 404 for missing resources, and 500 for unexpected server/database errors.

5. API endpoint inventory

METHOD PATH	ACCESS	PURPOSE
POST /api/register	Public	Create student account; JSON studentId, email, password, confirmPassword.
POST /api/login	Public	Student login; JSON studentId and password.
POST /api/admin/login	Public	Administrator login; JSON email and password.
POST /api/logout	Any visitor	Clear current session cookie.
GET /api/me	Student	Current student personal record.
GET /api/my-courses	Student	Current student courses, enrolments, lecturers.
GET /api/my-fees	Student	Current student fees, payments, balances.
GET /api/my-payments	Student	Current student payment history.

GET /api/my-outstanding-fees Student Outstanding-fee results and total.
GET /api/students Admin All student personal records.
GET /api/students/{student_id} Student owner/Admin One student personal record.
GET /api/students/{student_id}/courses Student owner/Admin One student enrolments.
GET /api/students/{student_id}/fees Student owner/Admin One student fees and balance.
GET /api/students/{student_id}/payments Student owner/Admin One student payment history.
GET /api/students/{student_id}/outstanding-fees Student owner/Admin One student outstanding-fee results.
GET /api/courses Public Course catalogue.
GET /api/courses/{course_id}/lecturer Public Course-to-lecturer assignment(s).
GET /api/lecturers Public Lecturer directory.
GET /api/lecturers/{lecturer_id}/teaching-assistants Public Lecturer-to-TA assignment(s).
GET /api/teaching-assistants Public Teaching-assistant directory.
GET /api/outstanding-fees Admin Department-wide outstanding-fee results.
GET /api/admin/stats Admin Department totals.
GET /api/admin/students?search= Admin Student directory/balances; search optional.
GET /api/admin/students/{student_id} Admin Student detail, courses, fees, payments.
GET /api/admin/resources/{resource} Admin Whitelisted department resource table.
GET /api/test-db Admin Protected connection diagnostic.
student_id and course_id must be non-empty alphanumeric IDs (hyphens permitted). lecturer_id must be a positive integer. The resource name is limited to a fixed query whitelist.

6. Authentication, authorization, and security

- Passwords are hashed/verified with bcrypt; API responses do not expose passwords or password hashes.
- Login produces an HMAC-signed HTTP-only session cookie with an eight-hour lifetime, sameSite=lax, and secure enabled in production.
- The token contains only role, optional student ID, and expiry; SESSION_SECRET is required and kept in environment configuration.
- requireStudent, requireAdmin, and requireStudentOrAdmin enforce role and student-record ownership.
- Student-only routes derive the student ID from the signed session. Department-wide balance and diagnostic routes require admin.
- SQL uses parameter placeholders with pg. Dynamic admin resource selection uses a fixed in-code query whitelist.
- Database environment values are not returned, and database errors are reported through generic server-error JSON.

7. Database scripts and backup

- database/create_tables.sql: exported schema, constraints, sequences, and outstanding-fee functions.
- database/insert_data.sql: project data inserts.
- database/functions.sql: standalone outstanding-fee function definitions.
- database/backup.sql: database export/backup for submission or restoration reference.

8. Next.js application and dashboard

The existing authenticated student portal uses /api/me, /api/my-courses, /api/my-fees, /api/my-payments, and /api/my-outstanding-fees. It includes profile, courses/enrolments, fees, payments, outstanding-balance pages, and a dashboard summary. The administrator interface includes a dashboard, searchable student directory, per-student detail page, department statistics, and resource tables. The Project 2 API work preserves these interfaces.

9. Testing and verification performed

- Inspected all API route files, shared database/auth helpers, database schema/functions, UI API consumers, and README.
- node --check passed for every route.js API file.
- git diff --check completed without whitespace errors.
- npm.cmd run build reached Next.js 'Compiled successfully'. The environment execution limit interrupted the later lint/type-check stage; no error was reported before timeout.
- A separate ESLint invocation also timed out without diagnostics. This report therefore does not claim a completed lint pass.

The README contains example PowerShell requests and invalid/unknown-ID cases. Full live endpoint testing requires the configured PostgreSQL instance and valid credentials; no unavailable live-test results are claimed here.

10. Assumptions and conclusion

- The database has been loaded/restored from the supplied scripts and .env.local contains valid connection values.

- The existing public course, lecturer, TA, and assignment directory routes remain public; sensitive student and department-wide data routes are role-protected.
- The implemented API provides read access for Project 1 functions plus account/session operations; browser management CRUD is not claimed.

Project 2 delivers a JSON web service over the Project 1 PostgreSQL Student Management System. All five required functional areas are exposed through APIs. The implementation combines role/ownership checks, signed HTTP-only sessions, bcrypt, parameterized SQL, controlled errors, and included database scripts/backup. It is ready for live testing with its configured PostgreSQL environment and repository submission.